

Q2 and Q21 Analysis:

In the compliance paper, the problem with **Q2** is that the **View** takes more time than **pure RLS**. In **Q21**, there is a timeout issue with **Views**, so the reasons need to be investigated.

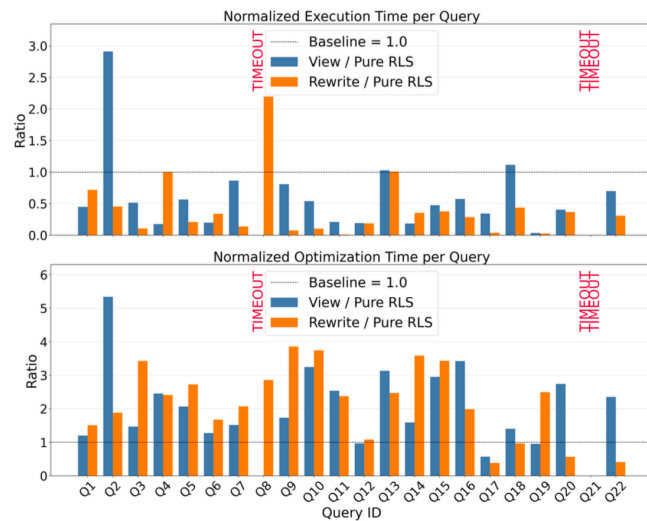


Figure 2: RLS vs. View vs. Query Rewrite for Acyclic Policies

Q2:

Tested on the **TPC-H 500 MB dataset** generated using **dbgen**.

Policies: (Source:

https://github.com/imtiyazuddin/secure26db/blob/main2/src/postgreSQL/acyclic_policy_query_mapping.json)

supplier:

```
CREATE POLICY supplier_rls_policy
ON supplier
USING (
    s_acctbal > 0
    AND EXISTS (
        SELECT 1
        FROM nation n,
             region r
        WHERE n.n_nationkey = supplier.s_nationkey
              AND n.n_regionkey = r.r_regionkey
              AND r.r_name = 'EUROPE'
    )
);
```

partsupp:

```

CREATE POLICY partsupp_rls_policy
ON partsupp
USING (
    ps_availqty > 100
    AND EXISTS (
        SELECT 1
        FROM supplier s
        WHERE s.s_suppkey = partsupp.ps_suppkey
            AND s.s_acctbal > 0
    )
);

```

Query:

```

select
    s_acctbal,
    s_name,
    n_name,
    p_partkey,
    p_mfgr,
    s_address,
    s_phone,
    s_comment
from
    part,
    supplier,
    partsupp,
    nation,
    region
where
    p_partkey = ps_partkey
    and s_suppkey = ps_suppkey
    and p_size = 15
    and p_type like '%BRASS'
    and s_nationkey = n_nationkey
    and n_regionkey = region.r_regionkey
    and region.r_name = 'EUROPE'
    and ps_supplycost = (
        select
            min(ps_supplycost)
        from
            partsupp,
            supplier,
            nation,
            region
        where
            p_partkey = ps_partkey

```

```

        and s_suppkey = ps_suppkey
        and s_nationkey = n_nationkey
        and n_regionkey = region.r_regionkey
        and region.r_name = 'EUROPE'
    )
order by
    s_acctbal desc,
    n_name,
    s_name,
    p_partkey
limit 100;

```

I ran the **RLS** and **View** versions 10 times each and calculated the average performance for RLS.

```

RLS: (34.645525 seconds)
Runs: 10
Average Planning Time: 31.538 ms
Average Execution Time: 34645.525 ms

```

```

Views: (21.679378 seconds)
Runs: 10
Average Planning Time: 37.936 ms
Average Execution Time: 21679.378 ms

```

The **View's planning time is greater than the RLS planning time, as usual, and the View is faster than RLS.**

So, first, let's analyze how the transformations of **RLS** and **Views** differ.

Link to the Annotated logs where we have transformed logs only:

RLS: <https://github.com/adityadave29/PE/blob/main/PE-Daywise-Work-Tracker/19-22-JUNE/Q2%26Q21/Q2/both-partsupp-supplier/rls/transformation-part-of-rls-only.pdf>

VIEW: <https://github.com/adityadave29/PE/blob/main/PE-Daywise-Work-Tracker/19-22-JUNE/Q2%26Q21/Q2/both-partsupp-supplier/views/transformation-part-of-view-only.pdf>

RLS after pulling stage:

```

PROPARALLEL_SAFE
WARNING: is_parallel_safe: INPUT qual = ((RESTRICTINFO :clause (OEXPR :opno 1756 :opfuncid 1720 :opresulttype 16 :opretset false :opcollid 0 :inputcollid 0 :args ((VAR :varno 2 :varrattno 2 :vartype 17
00 :vartypmod 983846 :varcollid 0 :varnullingrels (b) :varlevelsup 0 :varreturningtype 0 :varnosyn 2 :varattosyn 6 :location -1) (CONST :consttype 1700 :consttypmod -1 :constcollid 0 :constlen -1 :cons
tbyval false :constisnull false :location -1 :constvalue 6 ( 24 0 0 0 0 -128 ))) :location -1) :is_pushed_down true :can_join false :pseudoconstant false :has_clone false :is_clone false :leakproof false
:has_volatile 0 :security_level 0 :num_base_rels 1 :clause_relds (b 2) :required_relds (b 2) :incompatible_relds (b) :outer_relds (b) :left_relds (b 2) :right_relds (b) :orclause <> :rinfo_serial
1 :eval_cost.startup -1 :eval_cost.per_tuple 0 :norm_select -1 :outer_select -1 :mergopfamilies <> :left_em <> :right_em <> :outer_is_left false :hashjoinoperator 0 :left_bucketsize -1 :right_bucketsiz
e -1 :left_mcvfreq -1 :right_mcvfreq -1 :left_hashoperator 0 :right_hashoperator 0) (RESTRICTINFO :clause (ALTERNATIVESUBPLAN :subplans ((SUBPLAN :sublinkType 0 :testexpr <> :paramsid <> :plan_id 10
:plan_name SubPlan18 :firstColType 2278 :firstColTypmod -1 :firstColCollation 0 :useHashTable false :unknownEqFalse true :parallel_safe false :setParameter <> :parParam (i 9) :args ((VAR :varno 2 :varrat
tno 4 :vartype 23 :vartypmod -1 :varcollid 0 :varnullingrels (b) :varlevelsup 0 :varreturningtype 0 :varnosyn 2 :varattosyn 2 :location -1) :startup_cost 0 :per_call_cost 24.2626) (SUBPLAN :sublinkType
2 :testexpr (OEXPR :opno 96 :opfuncid 65 :opresulttype 16 :opretset false :opcollid 0 :inputcollid 0 :args ((VAR :varno 3 :varrattno 2 :vartype 23 :vartypmod -1 :varcollid 0 :varnullingrels (b) :varle
velsup 0 :varreturningtype 0 :varnosyn 4 :location -1) (PARAM :paramkind 1 :paramid 10 :paratypmod 23 :paratypmod -1 :paramcollid 0 :location -1)) :location -1) :paramsid (i 10) :plan_id 1
:plan_name SubPlan11 :firstColType 23 :firstColTypmod -1 :firstColCollation 0 :useHashTable true :unknownEqFalse true :parallel_safe true :setParameter <> :parParam <> :args <> :startup_cost 24.4875 :pe
r_call_cost 0.0025))) :is_pushed_down true :can_join false :pseudoconstant false :has_clone false :is_clone false :leakproof false :has_volatile 0 :security_level 0 :num_base_rels 1 :clause_relds (b 2)
:required_relds (b 2) :incompatible_relds (b) :outer_relds (b) :left_relds (b) :right_relds (b) :orclause <> :rinfo_serial 2 :eval_cost.startup -1 :eval_cost.per_tuple 0 :norm_select -1 :outer_sele
c -1 :mergopfamilies <> :left_em <> :right_em <> :outer_is_left false :hashjoinoperator 0 :left_bucketsize -1 :right_bucketsize -1 :left_mcvfreq -1 :right_mcvfreq -1 :left_hashoperator 0 :right_hash
operator 0))
WARNING: max_parallel_hazard_walker: SubPlan plan_id=10 parallel_safe=0 > reason=correlated_subplan
WARNING: is_parallel_safe: OUTPUT qual = ((RESTRICTINFO :clause (OEXPR :opno 1756 :opfuncid 1720 :opresulttype 16 :opretset false :opcollid 0 :inputcollid 0 :args ((VAR :varno 2 :varrattno 2 :vartype 1
700 :vartypmod 983846 :varcollid 0 :varnullingrels (b) :varlevelsup 0 :varreturningtype 0 :varnosyn 2 :varattosyn 6 :location -1) (CONST :consttype 1700 :consttypmod -1 :constcollid 0 :constlen -1 :cons
tbyval false :constisnull false :location -1 :constvalue 6 ( 24 0 0 0 0 -128 ))) :location -1) :is_pushed_down true :can_join false :pseudoconstant false :has_clone false :is_clone false :leakproof false
:has_volatile 0 :security_level 0 :num_base_rels 1 :clause_relds (b 2) :required_relds (b 2) :incompatible_relds (b) :outer_relds (b 2) :right_relds (b) :orclause <> :rinfo_serial
1 :eval_cost.startup -1 :eval_cost.per_tuple 0 :norm_select -1 :outer_select -1 :mergopfamilies <> :left_em <> :right_em <> :outer_is_left false :hashjoinoperator 0 :left_bucketsize -1 :right_bucketsiz
e -1 :left_mcvfreq -1 :right_mcvfreq -1 :left_hashoperator 0 :right_hashoperator 0) (RESTRICTINFO :clause (ALTERNATIVESUBPLAN :subplans ((SUBPLAN :sublinkType 0 :testexpr <> :paramsid <> :plan_id 1
0 :plan_name SubPlan10 :firstColType 2278 :firstColTypmod -1 :firstColCollation 0 :useHashTable false :unknownEqFalse true :parallel_safe false :setParameter <> :parParam (i 9) :args ((VAR :varno 2 :varrat
tno 4 :vartype 23 :vartypmod -1 :varcollid 0 :varnullingrels (b) :varlevelsup 0 :varreturningtype 0 :varnosyn 2 :varattosyn 2 :location -1) :startup_cost 0 :per_call_cost 24.2626) (SUBPLAN :sublinkType
2 :testexpr (OEXPR :opno 96 :opfuncid 65 :opresulttype 16 :opretset false :opcollid 0 :inputcollid 0 :args ((VAR :varno 2 :varrattno 2 :vartype 23 :vartypmod -1 :varcollid 0 :varnullingrels (b) :varle
velsup 0 :varreturningtype 0 :varnosyn 2 :varattosyn 4 :location -1) (PARAM :paramkind 1 :paramid 10 :paratypmod 23 :paratypmod -1 :paramcollid 0 :location -1)) :location -1) :paramsid (i 10) :plan_id 1
:plan_name SubPlan11 :firstColType 23 :firstColTypmod -1 :firstColCollation 0 :useHashTable true :unknownEqFalse true :parallel_safe true :setParameter <> :parParam <> :args <> :startup_cost 24.4875 :pe
r_call_cost 0.0025))) :is_pushed_down true :can_join false :pseudoconstant false :has_clone false :is_clone false :leakproof false :has_volatile 0 :security_level 0 :num_base_rels 1 :clause_relds (b 2)
:required_relds (b 2) :incompatible_relds (b) :outer_relds (b) :left_relds (b) :right_relds (b) :orclause <> :rinfo_serial 2 :eval_cost.startup -1 :eval_cost.per_tuple 0 :norm_select -1 :outer_sele
c -1 :mergopfamilies <> :left_em <> :right_em <> :outer_is_left false :hashjoinoperator 0 :left_bucketsize -1 :right_bucketsize -1 :left_mcvfreq -1 :right_mcvfreq -1 :left_hashoperator 0 :right_hash
operator 0)) | parallel_safe = false | reason = PROPARALLEL_RESTRICTED
WARNING: is_parallel_safe: INPUT qual = ((RESTRICTINFO :clause (OEXPR :opno 521 :opfuncid 147 :opresulttype 16 :opretset false :opcollid 0 :inputcollid 0 :args ((VAR :varno 3 :varrattno 3 :vartype 23 :
vartypmod -1 :varcollid 0 :varnullingrels (b) :varlevelsup 0 :varreturningtype 0 :varnosyn 3 :varattosyn 3 :location -1) (CONST :consttype 23 :consttypmod -1 :constcollid 0 :constlen 4 :constbyval true
:constisnull false :location -1 :constvalue 4 ( 100 0 0 0 0 0 0 1)) :location -1) :is_pushed_down true :can_join false :pseudoconstant false :has_clone false :is_clone false :leakproof false :has_vol
atile 0 :security_level 0 :num_base_rels 1 :clause_relds (b 3) :required_relds (b 3) :incompatible_relds (b) :outer_relds (b 3) :right_relds (b 3) :orclause <> :rinfo_serial 3 :eval_
cost.startup -1 :eval_cost.per_tuple 0 :norm_select -1 :outer_select -1 :mergopfamilies <> :left_em <> :right_em <> :outer_is_left false :hashjoinoperator 0 :left_bucketsize -1 :right_bucketsize -1 :left_
_mcvfreq -1 :right_mcvfreq -1 :left_hashoperator 0 :right_hashoperator 0) (RESTRICTINFO :clause (ALTERNATIVESUBPLAN :subplans ((SUBPLAN :sublinkType 0 :testexpr <> :paramsid <> :plan_id 14 :plan_nam
e SubPlan14 :firstColType 2278 :firstColTypmod -1 :firstColCollation 0 :useHashTable false :unknownEqFalse true :parallel_safe false :setParameter <> :parParam (i 11) :args ((VAR :varno 3 :varrattno 2 :var
type 23 :vartypmod -1 :varcollid 0 :varnullingrels (b) :varlevelsup 0 :varreturningtype 0 :varnosyn 3 :varattosyn 2 :location -1) :startup_cost 0 :per_call_cost 121512.000000000003) (SUBPLAN :sublinkType
2 :testexpr (OEXPR :opno 96 :opfuncid 65 :opresulttype 16 :opretset false :opcollid 0 :inputcollid 0 :args ((VAR :varno 3 :varrattno 2 :vartype 23 :vartypmod -1 :varcollid 0 :varnullingrels (b) :varle
velsup 0 :varreturningtype 0 :varnosyn 3 :varattosyn 2 :location -1) (PARAM :paramkind 1 :paramid 14 :paratypmod 23 :paratypmod -1 :paramcollid 0 :location -1)) :location -1) :paramsid (i 14) :plan_id 1
7 :plan_name SubPlan17 :firstColType 23 :firstColTypmod -1 :firstColCollation 0 :useHashTable true :unknownEqFalse true :parallel_safe false :setParameter <> :parParam <> :args <> :startup_cost 121502.6
2500000001 :per_call_cost 0.0025))) :is_pushed_down true :can_join false :pseudoconstant false :has_clone false :is_clone false :leakproof false :has_volatile 0 :security_level 0 :num_base_rels 1 :clau
se_relds (b 3) :required_relds (b 3) :incompatible_relds (b) :outer_relds (b) :left_relds (b) :right_relds (b) :orclause <> :rinfo_serial 4 :eval_cost.startup -1 :eval_cost.per_tuple 0 :norm_select
-1 :outer_select -1 :mergopfamilies <> :left_em <> :right_em <> :outer_is_left false :hashjoinoperator 0 :left_bucketsize -1 :right_bucketsize -1 :left_mcvfreq -1 :right_mcvfreq -1 :left_hashoperator
0 :right_hashoperator 0)) | parallel_safe = false | reason = PROPARALLEL_RESTRICTED
WARNING: is_parallel_safe: INPUT qual = <>

```

As usual, the policies are restricted from parallelism. Also, in the main query, the subquery is converted into a plan, and its parallel_safe flag is set to false, possibly because the supplier and partsupp tables have RLS policies. Therefore, this is what I observe at the pull-up stage.

```

select
min(ps_supplycost)
from
partsupp,
supplier,
nation,
region
where
p_partkey = ps_partkey
and s_suppkey = ps_suppkey
and s_nationkey = n_nationkey
and n_regionkey = region.r_regionkey
and region.r_name = 'EUROPE'
)

```

In the views:

Subqueries are converted to joins, and only the main query's subquery is converted into a SUBPLAN; everything else is optimized. Therefore, I think the View should be faster than RLS.

I also checked again these policies are same as mentioned here:

https://github.com/imtiyazuddin/secure26db/blob/main2/src/postgreSQL/acyclic_policy_query_mapping.json

I tested Q2 on a 1 GB dataset as well.

So, the behavior remains the same: planning time (VIEW) > planning time (RLS), and execution time (VIEW) < execution time (RLS).

RLS:

Planning Time: 37.931 ms

Execution Time: 680918.007 ms

VIEW:

Planning Time: 41.099 ms

Execution Time: 584614.410 ms

Q21:

Performed this on the 300 MB TPC-H dataset generated using dbgen.

RLS:

Runs: 5

Average Planning Time: 28.394 ms

Average Execution Time: 1233.670 ms

VIEW:

Runs: 5

Average Planning Time: 29.635 ms

Average Execution Time: 495.383 ms

Policy 1:

```
CREATE POLICY supplier_rls_policy
ON supplier
USING (
    s_acctbal > 0
    AND EXISTS (
        SELECT 1
        FROM nation n,
             region r
        WHERE n.n_nationkey = supplier.s_nationkey
              AND n.n_regionkey = r.r_regionkey
              AND r.r_name = 'EUROPE'
    )
);
```

Policy 2:

```

CREATE POLICY lineitem_rls_policy
ON lineitem
USING (
    EXISTS (
        SELECT 1
        FROM orders o
        WHERE o.o_orderkey = lineitem.l_orderkey
            AND o.o_orderpriority IN ('1-URGENT', '2-HIGH')
    )
    AND EXISTS (
        SELECT 1
        FROM partsupp ps
        WHERE ps.ps_partkey = lineitem.l_partkey
            AND ps.ps_suppkey = lineitem.l_suppkey
            AND ps.ps_availqty > 0
    )
);

```

```

select
    s_name,
    count(*) as numwait
from
    supplier,
    lineitem l1,
    orders,
    nation
where
    s_suppkey = l1.l_suppkey
    and o_orderkey = l1.l_orderkey
    and o_orderstatus = 'F'
    and l1.l_receiptdate > l1.l_commitdate
    and exists (
        select *
        from lineitem l2
        where l2.l_orderkey = l1.l_orderkey
            and l2.l_suppkey <> l1.l_suppkey
    )
    and not exists (
        select *
        from lineitem l3
        where l3.l_orderkey = l1.l_orderkey
            and l3.l_suppkey <> l1.l_suppkey
            and l3.l_receiptdate > l3.l_commitdate
    )
    and s_nationkey = n_nationkey
    and n_name = 'ARGENTINA'

```

```
group by
    s_name
order by
    numwait desc,
    s_name;
```

So, the same thing is happening here: the securityQuals remain unchanged, while the View conditions are merged using joins. As a result, Views are faster than RLS. I also checked the policies, and they are the same policies applied to Query 21 in the compliance paper.

I am using PostgreSQL 18.4, whereas the paper mentions that it used PostgreSQL 18.2.

From this analysis, both Queries Q2 and Q21 are behaving normally.

Logs: <https://github.com/adityadave29/PE/tree/main/PE-Daywise-Work-Tracker/19-22-JUNE>